

REPASO INTEGRADOR · PARTE 0 · INTRO

Programación Orientada a Objetos en C++

Repaso Pre-Parcial

Módulo 0 — Fundamentos técnicos (ejemplos genéricos, sin el caso de estudio)

```
class ObjetoDeEstudio {  
private:  
    Conocimiento conocimiento;  
public:  
    void prepararParcial();  
};
```

Memoria Heap y Swap · Vector dinámico · Cadenas tipo C · Recursividad

Programación Orientada a Objetos - Leonardo Brambilla - FCyT - UADER

MÓDULO

0

≈ 20 minutos

Fundamentos técnicos previos

Objetivos de esta sección

- Repasar, con ejemplos genéricos (sin el caso de estudio), cuatro mecánicas de bajo nivel de C++.
- Estas ideas se aplicarán luego al Sistema de Biblioteca en los módulos correspondientes.

Memoria: Stack vs. Heap

En C++, distinguiremos principalmente dos regiones de memoria:

- ▶ **Stack (pila)**: memoria automática, administrada por el compilador.
- ▶ **Heap (montículo)**: memoria dinámica, administrada por el programador

- ▶ El **heap** (o free store) es la región de RAM donde vive la memoria dinámica: la reserva el programador con `new` y la libera con `delete`.
- ▶ A diferencia del **stack**, el heap NO se organiza solo: si no liberamos lo reservado, esa memoria queda ocupada aunque ya no la usemos (memory leak).
- ▶ Cuando la RAM disponible se agota, el sistema operativo puede usar memoria swap: espacio del disco que actúa como extensión de la RAM.
- ▶ Acceder a swap es mucho más lento que acceder a RAM; por eso gestionar bien el heap (liberar lo que no se usa) importa para el rendimiento.

RAM vs. Swap

RAM (rápida)		DISCO (lento)
+-----+		+-----+
Stack		
(automático)		SWAP
+-----+		(extensión de
Heap	<---->	la RAM cuando
(new / delete)		ésta se llena)
+-----+		+-----+

`new / delete` manejan el HEAP.

El SO decide, por detrás, cuándo usar SWAP si falta RAM.

? ¿Por qué acceder a memoria en swap (disco) es mucho más lento que acceder a memoria en RAM?

Vector dinámico: crear y ampliar directamente

- ▶ Un arreglo reservado con `new T[n]` tiene tamaño fijo: no se puede simplemente 'agregar' un lugar más.
- ▶ Estrategia directa (sin capacidad extra): para agregar UN elemento, se crea un arreglo nuevo de tamaño `actual+1`, se copian los valores existentes, se agrega el nuevo y se libera el arreglo viejo.
- ▶ Es la forma más simple de entender el crecimiento dinámico: cada inserción reserva memoria nueva del tamaño exacto que se necesita.
- ▶ Más adelante veremos esta misma idea aplicada al catálogo de libros de la Biblioteca.

vector dinámico genérico (int)

```
int* datos = new int[0]; // vacío
int cantidad = 0;

void agregar(int valor) {
    int* nuevo = new int[cantidad + 1];
    for (int i = 0; i < cantidad; i++)
        nuevo[i] = datos[i];
    nuevo[cantidad] = valor;
    delete[] datos; // libera el arreglo viejo
    datos = nuevo; // el nuevo pasa a ser el actual
    cantidad++;
}
```

Memoria al agregar el valor 9

```
ANTES: [5][2][7]  cantidad=3
agregar(9)
DESPUÉS: [5][2][7][9]  cantidad=4
(arreglo NUEVO, el viejo de 3 se liberó)
```

 **Error frecuente:** Olvidar `delete[] datos;` antes de reasignar `'datos = nuevo;'`, lo que provoca un memory leak en cada inserción.

 ¿Qué costo tiene esta estrategia si agrego 100 elementos uno por uno? ¿Cuántas copias completas del arreglo se realizan en total?

Vector dinámico: eliminar un elemento

- ▶ Eliminar también implica crear un arreglo nuevo, esta vez de tamaño cantidad-1.
- ▶ Se copian todos los valores existentes EXCEPTO el que está en la posición a eliminar.
- ▶ Luego se libera el arreglo viejo con delete[] y el nuevo pasa a ser el vector actual.
- ▶ La misma idea se aplicará más adelante para eliminar un Libro del catálogo de la Biblioteca.

vector dinámico genérico — eliminar

```
void eliminar(int indice) {  
    int* nuevo = new int[cantidad - 1];  
    int j = 0;  
    for (int i = 0; i < cantidad; i++)  
        if (i != indice)  
            nuevo[j++] = datos[i];  
    delete[] datos;    // libera el arreglo viejo  
    datos = nuevo;  
    cantidad--;  
}
```

Memoria al eliminar el índice 1

```
ANTES: [5][2][7][9]  cantidad=4  
eliminar(1)  
DESPUÉS: [5][7][9]  cantidad=3  
(arreglo NUEVO de 3, sin el valor en el índice 1)
```

 **Error frecuente:** No validar que 'indice' esté dentro del rango [0, cantidad-1] antes de eliminar, provocando acceso fuera de los límites del arreglo.

 ¿Qué debería hacer la función si 'indice' es negativo o mayor o igual a 'cantidad'?

Cadenas tipo C: funciones de <cstring>

- ▶ `char[]` es un arreglo de caracteres terminado en `'\0'`; no crece solo y no tiene operadores como `+`, `==` o `<<`.
- ▶ La biblioteca `<cstring>` ofrece funciones para trabajar con estos arreglos: `strlen`, `strcpy`, `strcmp`, `strcat`.
- ▶ También existe `std::string`, que veremos más adelante; por ahora el foco está en `char[]` y sus funciones.

cstring genérico

```
#include <cstring>

char a[20] = "hola";
char b[20];

strcpy(b, a);          // copia "hola" en b
cout << strlen(b);     // longitud: 4
cout << strcmp(a,b);   // 0 -> son iguales
strcat(b, "!!!");      // b = "hola!!!"
```

 **Error frecuente:** Comparar `char[]` con `'=='` en vez de `strcmp`: `'=='` compara direcciones de memoria, no el contenido.



De este tema pueden tener machete, no hace falta que se lo memoricen.

? ¿Qué imprime `'cout << (a == b);'` con las variables del ejemplo, aunque ambas contengan el texto "hola"?

Recursividad: caso base y caso recursivo

- ▶ Una función es recursiva cuando se llama a sí misma para resolver una versión más pequeña del mismo problema.
- ▶ Caso base: condición simple que se resuelve sin volver a llamarse; detiene la recursión.
- ▶ Caso recursivo: reduce el problema y confía en que la llamada recursiva resuelve esa versión más chica.
- ▶ Se profundiza con más ejemplos y el árbol de llamadas en el módulo dedicado a Recursividad.

factorial genérico

```
int factorial(int n) {  
    if (n == 0) return 1;      // caso base  
    return n * factorial(n-1); // caso recursivo  
}  
  
factorial(4)  
= 4 * factorial(3)  
= 4 * (3 * factorial(2))  
= 4 * (3 * (2 * factorial(1)))  
= 4 * (3 * (2 * (1 * factorial(0))))  
= 24
```

 **Error frecuente:** Omitir el caso base, o escribir un caso recursivo que no se acerca a él: provoca recursión infinita y stack overflow.

? ¿Qué pasaría si `factorial(n)` llamara a `factorial(n)` en vez de `factorial(n-1)`?